COMPETITION BUILD / 02 NOVEMBER 2026

SuperChat

Phase 1 technical report

1. Architecture and scope

SuperChat combines flights, shopping and cinema behind one responsive web chat. The implementation uses FastAPI, a buildless HTML/CSS/JavaScript interface, domain modules and the official JSON data. No real payment or booking provider is contacted. The reference clock is fixed to 2 November 2026 (Asia/Baku), as required by the brief. Flight times remain airport-local; durations come directly from the supplied records.

The request flow is: text / speech transcript / image category > router > domain operations > repository validation > grounded response > durable session event. Mixed requests call each relevant module and return labeled sections in one reply. The browser offers saved conversations, product/flight/showtime cards, comparisons, cart and booking summaries, log export and explicit context continuation.

2. Routing and response grounding

The offline router uses normalized English, Azerbaijani and Russian vocabulary, domain-specific IDs, date parsers and current conversation state. It recognizes the seven brief categories, plus a greeting/help category. Domain-specific clauses are separated for mixed requests. Ordinals and cheaper-option references resolve against the previous result ordering. Unclear references after a mixed request ask a short clarification rather than selecting an arbitrary domain.

Optional OpenRouter inference translates flexible language into a Pydantic-validated plan: domain, action, filters, quantity and selected IDs. It receives recorded state and recent messages, with the fixed reference date. JSON mode works across providers; independent validation rejects invalid shapes, action/domain combinations, quantities and dates. Provider timeout/invalid output falls back to offline parsing and is visible in the reply.

The model never writes catalogue facts or final reservation confirmations. The application selects actual repository records and constructs prices, availability, product details and totals deterministically. Model-suggested IDs must be explicit in the request or present in recent results. Mutating model actions also require an explicit action in the original request. There are no arbitrary code, SQL, shell or web-search tools. This sacrifices open-ended prose for auditable factual responses.

3. Conversation state and log durability

Each chat has a canonical UUID and an append-only JSONL file. The first event creates the session. Each later event atomically records the user message, grounded assistant reply, routing trace, domain operations and a complete post-turn state. State includes language, explicit preferences, active domains, per-domain filters/results/selection, cart, reservations and pending clarification. No generated memory summary is authoritative.

Resuming a chat replays the last recorded state and complete transcript. New chats start independently. An explicit "new chat with this context" operation copies search context and preferences with provenance, excluding cart, reservations and pending actions. Historical transcript facts remain historical; current operations re-read source data.

An OS file lock serializes read/validate/append across processes. Remaining seats are official availability minus all logged reservations. A completed turn is appended and fsynced once, including mixed-domain operations; source errors discard the tentative state. Request UUIDs provide idempotency: a replay returns the original response, preventing double booking after a lost connection. Corrupt logs fail closed instead of silently dropping inventory events. This is suitable for a local demo; replaying all sessions and holding the lock during hosted interpretation does not scale to a distributed service.

4. Repository boundary and domain behaviour

FlightRepository, ProductRepository and ShowtimeRepository define search/get interfaces with JSON implementations. Cinema records join showtimes to their exact movie and cinema IDs. Search includes route/date, price/stops/departure/airline for flights; text/category/brand/price/colour for products; and genre/rating/language/date/cinema for film sessions. Details and comparisons expose source record IDs.

Reservations validate a single exact flight/showtime and quantity against remaining seats. Cart operations check current product stock and calculate totals locally; cart contents do not reserve inventory. No source JSON is edited. Missing matches are stated explicitly. Phase 2 can inject real read adapters, but real write operations must also use provider reservation APIs, idempotency, authentication and authoritative inventory transactions.

5. Bonus implementation

Voice uses the browser Web Speech API in continuous mode. After one microphone enable, speech start/end events and a pause timer submit final transcripts through the same route. Optional speech synthesis reads responses. Browser support, Azerbaijani locale availability and internet requirements vary; denied/unsupported recognition has a visible fallback.

Visual search validates JPEG/PNG/WebP input size and dimensions, strips metadata through image decoding, and classifies product type through OpenRouter vision or local CLIP. Ten catalogue types and an "other" category are considered. Local acceptance needs score >=0.65 and margin >=0.20; hosted acceptance needs >=0.75. Scores are uncalibrated model confidence, not correctness probabilities. Uncertainty asks a question. Accepted categories search the real catalogue; the system does not claim an exact brand identity. Image bytes are not retained. CLIP is cached for offline demos.

English, Azerbaijani and Russian are supported through translated UI labels, response templates, product descriptions, domain vocabulary and date/action parsing. Mid-session language changes preserve domain state. Optional hosted parsing extends coverage beyond the explicit rule vocabulary.

6. Validation, known limits and source decisions

The automated suite covers the brief's sample flows, domain returns, multilingual queries, comparisons, stock limits, cart totals, ambiguous references, current-data revalidation, malformed logs, restart recovery, session isolation, explicit continuation, duplicate requests, concurrent seat reservations, provider failure and image confidence paths. The offline router scores 102/106 (96.2%) on an author-written set. This is a regression result, not an independent generalization benchmark. Four harder indirect/slang queries fall back to clarification. Per-language figures and every input are in TEST_LOG.md.

A real headphones photo was recognized locally with score 0.9209 and margin 0.8443; a blank image was rejected at 0.2461. These two smoke tests do not measure broad vision accuracy. Browser visual and microphone testing were unavailable because the host denied Computer Use permission. No live OpenRouter test was possible without a user key. Full natural language and multilingual performance beyond the finite parser require broader evaluation. Optional round trips, real APIs/payments, public multi-user authentication and distributed persistence are outside this build.

The supplied LangChain router tutorial fits the architecture. The supplied RAG research template is document-retrieval oriented and archived; its indexing/vector-store dependencies do not help exact stock and reservation operations. LangGraph Supervisor was also reviewed, but autonomous handoff loops add unnecessary complexity here. None was cloned or copied; the implementation is purpose-built. Runtime dependencies are locked, with source and licence notes in REFERENCES.md.

References: LangChain router tutorial; langchain-ai/rag-research-agent-template; langchain-ai/langgraph-supervisor-py; OpenRouter structured output and image documentation; Hugging Face zero-shot classification/CLIP model card; MDN Web Speech API. Full linked citations and the undistributed image-test attribution are in REFERENCES.md.

Router pattern

RAG research template

LangGraph Supervisor

OpenRouter structured outputs

CLIP model and model card

Web Speech API